Functional vs Non-Functional Requirements: Class Notes

Oct 19, 2026

1. Introduction: Why two types of requirements?

Every software product is built to answer two questions: **"What should the system do?"** (functional) and **"How well should it do it?"** (non-functional).

A **requirement** is a statement of what the customer or user needs from the software. Before developers write code and before testers write test cases, requirements are collected and written down, usually in a document called the SRS (Software Requirement Specification).

Simple analogy: buying a car

- Functional: the car must start, move forward, reverse, brake, and turn on the AC.
- Non-functional: it should give 20 km/litre mileage, reach 100 km/h in 10 seconds, be safe in a crash, and feel comfortable on long drives.

A car that has all the features but is unsafe, slow, or uncomfortable will still be rejected by the customer. Software works the same way.

Why students must know this

- It is the first step of every project (SDLC) and the base for writing test cases.
- Many project failures happen because of missed non-functional requirements, not missing features.
- It is a very common interview question for testers, developers, and business analysts.

2. Functional Requirements (FR)

Definition: Functional requirements describe *what the system must do*: the features, functions, and behaviour the user can see and use. If a feature is missing, the product is incomplete.

Key points

- Describe actions: input → processing → output.

- Written as "The system shall..." or "The user can...".
- Directly testable with pass/fail results.
- Defined by business needs and user stories.

Real-time examples

Application	Functional requirements
Amazon / Flipkart	User can search products; add items to cart; apply a coupon; pay by card, UPI, or COD; track the order
UPI apps (PhonePe, GPay)	User can register with mobile number; link a bank account; send money using UPI ID; check balance; view transaction history
IRCTC	User can log in; search trains by source, destination, and date; select class; book a ticket; cancel a ticket and receive a refund
WhatsApp	User can send text, image, and voice messages; create groups; make voice and video calls; delete a message
ATM machine	Insert card; enter PIN; select withdrawal; dispense cash; print receipt
Online exam portal (relevant for our mock tests)	Student logs in; starts the test; answers MCQs; submits; the system calculates and shows the score and leaderboard

Example of a well-written functional requirement

> "When a registered user enters a valid username and password and clicks Login, the system shall open the user's dashboard. If the password is wrong, the system shall display the message 'Invalid credentials'."

Notice that it is clear, specific, and can be tested.

3. Non-Functional Requirements (NFR)

Definition: Non-functional requirements describe *how well the system works*: its quality, performance, and the constraints it must respect. They do not add a new feature; they decide whether users are happy with the features.

Key points

- Also called quality attributes or "-ilities".
- Must be measurable. "The app should be fast" is weak. "The page must load in under 3 seconds for 10,000 users" is correct.
- Often affect the whole system, not one screen.
- Hard and costly to fix late in the project.

Main types with real-time examples

Type	What it means	Real-time example
Performance	Speed and response time	Google search results appear in under 1 second; a UPI payment completes in under 5 seconds
Scalability	Handles growth in users or data	Flipkart Big Billion Days or Amazon Prime Day: traffic jumps 10x and the site must not crash
Security	Protects data and access	OTP and PIN for banking; password stored in encrypted form; HTTPS; account locks after 3 wrong attempts
Availability	System is up when needed	IRCTC Tatkal booking at 10 AM must be working; banks target 99.9% uptime
Reliability	Works correctly without failure	An ATM must not deduct money without dispensing cash; failed transactions are auto-refunded
Usability	Easy to learn and use	A first-time user can book a cab in Ola/Uber within 3 taps; clear error messages; support for regional languages

Type	What it means	Real-time example
Compatibility	Works across devices, browsers, OS	A website works on Chrome, Edge, Safari, and on mobile and desktop
Maintainability	Easy to fix and update	A bug fix can be released without shutting down the whole app; clean, documented code
Portability	Can move to another platform	The same app runs on Android and iOS; moves from one cloud provider to another
Data integrity / Compliance	Follows rules and laws	Banking apps follow RBI guidelines; apps follow data-privacy laws

Example of a well-written non-functional requirement

> "The login page shall load within 2 seconds, even when 5,000 users are logging in at the same time."

This one is measurable: you can set a number, run a test, and say pass or fail.

4. Functional vs Non-Functional: Side by Side

Point	Functional	Non-Functional
Answers	What does the system do?	How well does it do it?
Focus	Features and behaviour	Quality and constraints
Source	Business needs, user stories, use cases	Standards, user expectations, technical limits
Testing type	Functional testing (system, integration, UAT)	Performance, security, usability, compatibility testing
Measured by	Works or does not work	Numbers: seconds, users, % uptime
If missing	Feature is absent; product incomplete	Feature exists but is slow, unsafe, or hard to use

Point	Functional	Non-Functional
Scope	Usually one feature or module	Usually the whole system
Example (online shopping)	Add to cart, pay, track order	Page loads in 2 sec; payment is secure; handles 1 lakh users
Example (ATM)	Withdraw cash	Completes in under 30 sec; PIN is encrypted; machine is rarely down

One-line memory trick: Functional = *verbs* (login, search, pay, send). Non-functional = *adjectives* (fast, secure, reliable, easy).

5. Worked Example: A Food Delivery App (Swiggy / Zomato style)

Use this one app to show students how both types work together.

Feature area	Functional requirement (what)	Non-functional requirement (how well)
Sign up / Login	User registers using mobile number and OTP	OTP arrives within 10 seconds; OTP expires in 5 minutes; password stored encrypted
Search	User searches restaurants by name or dish	Results appear in under 2 seconds, even with 50,000 restaurants listed
Order	User adds dishes to cart and places an order	App handles 2 lakh orders during the Sunday dinner rush without crashing
Payment	User pays by UPI, card, or cash on delivery	Payment data is secure; a failed payment is refunded automatically within the stated time
Tracking	User sees the delivery partner live on a map	Location updates every 5 seconds; battery use stays low
Support	User can raise a complaint	Works on Android, iOS, and web; supports English and Telugu or Hindi

Story to tell the class

Imagine this illustrative scenario (not a real incident): on New Year's Eve, a food app had all features working perfectly (functional passed). But the server could not handle the rush, the app hung at payment, and thousands of customers lost orders. Nothing was wrong with the *features*. The *non-functional* requirement, scalability, was ignored. This is why both types matter equally.

6. How Each Type Is Tested

Functional testing checks that every feature works as per the requirement. It is usually done first.

- Unit testing: one function or module (e.g. the "add to cart" function).
- Integration testing: modules working together (cart + payment).
- System testing: the full application end to end.
- Smoke and sanity testing: a quick check that the build is stable.
- Regression testing: old features still work after a new change.
- UAT (User Acceptance Testing): the customer confirms the product meets their needs.

Non-functional testing checks the quality attributes.

Testing type	What it checks	Real-time example
Performance testing	Speed under normal load	Does the IRCTC page load in 3 seconds?
Load testing	Behaviour at expected peak users	50,000 users booking at the same time
Stress testing	Breaking point beyond the limit	Keep adding users until the app crashes, then see how it recovers
Security testing	Attacks, data leaks, weak login	Try SQL injection; check that a password is not visible in the database
Usability testing	Ease of use	Ask a new user to place an order without help
Compatibility testing	Different browsers, devices, OS	Open the site on Chrome, Firefox, and an Android phone

Testing type	What it checks	Real-time example
Recovery / Reliability testing	Recovery after failure	Cut the network during payment; check whether money is refunded

Common tools students can hear about: Selenium and Playwright (functional automation), JMeter and LoadRunner (performance), Burp Suite and OWASP ZAP (security).

7. Class Activity, Quick Quiz, and Interview Questions

Activity 1: Sort the requirement (5 minutes)

Ask students to label each statement as Functional (F) or Non-Functional (NF).

1. User can reset a forgotten password using email. **(F)**
2. The app must open within 2 seconds. **(NF: performance)**
3. System shall allow 10,000 users at the same time. **(NF: scalability)**
4. Admin can download a monthly sales report. **(F)**
5. All card details must be encrypted. **(NF: security)**
6. The website must work on Chrome and Firefox. **(NF: compatibility)**
7. Customer can cancel an order before it ships. **(F)**
8. The system must be available 99.9% of the time. **(NF: availability)**

Activity 2: Group task (10 minutes)

Give each group one app (WhatsApp, Ola, BookMyShow, PhonePe). Ask them to write 3 functional and 3 non-functional requirements, with numbers for the non-functional ones.

Interview questions

1. What is the difference between functional and non-functional requirements?
2. Give two examples of each for a banking app.
3. Why should non-functional requirements be measurable?
4. What is the difference between load testing and stress testing?
5. Can a product pass all functional tests and still fail? Explain. (Yes. Poor performance, security, or usability can still make it fail.)
6. Is "the system should be user friendly" a good requirement? How would you improve it?

Key takeaways

- Functional = what the system does. Non-functional = how well it does it.
- Both are needed; a feature that is slow, unsafe, or unusable is as bad as a missing feature.
- Write non-functional requirements with numbers so they can be tested.
- Functional testing asks "does it work?" Non-functional testing asks "does it work well, safely, and under pressure?"